VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF APPLIED SCIENCE

GENERAL PHYSICS 1 - PH1003

ASSIGNMENT 1 - TOPIC 1

Determining Trajectory and Angular Momentum of Kinetic Motion

---

Advisor(s):	Assoc. Prof. Đỗ Ngọc Sơn	
Student(s):	Hắc Minh Quân	2550201
	Huỳnh Quốc Trọng Nghĩa	2550160
	Nguyễn Quốc Nam	2550156
	Văn Minh	2550153
	Nguyễn Doãn Nhân	2550173

HO CHI MINH CITY, NOVEMBER 2025

Member Allocation

No.	Member	Student ID	Task	Accomplishment
1	Hắc Minh Quân	2550201	Part 1,2,6,7,3.1 LaTex Coding	100%
2	Huỳnh Quốc Trọng Nghĩa	2550160	Part 3.2.1	100%
3	Nguyễn Quốc Nam	2550156	Part 4,5,6	100%
4	Văn Minh	2550153	Part 3.2.2	100%
5	Nguyễn Doãn Nhãn	2550173	Part 3.2.3	100%

Class: CC27 - Group: 01

Table of Contents

List of Figures

List of Tables

Listings

1 Introduction

1.1 Preface

The analysis of particle dynamics forms the cornerstone of classical mechanics, with applications spanning celestial orbits, projectile motion, and robotic path planning. A complete understanding of a particle's motion requires not only the knowledge of its trajectory, the path it traces in space, but also its dynamical quantities, such as angular momentum.

Angular momentum is a fundamental conserved quantity that provides deep insight into the rotational characteristics and constraints of a physical system, especially under central forces.

While analytical methods exist for solving these problems, computational approaches offer unparalleled efficiency, accuracy, and visualization capabilities. The ability to symbolically compute derivatives and vector operations, and to instantly visualize results, allows for rapid exploration and analysis of complex dynamical systems that would be cumbersome by hand.

1.2 Project Objectives

- To develop a MATLAB program that symbolically calculates the velocity and angular momentum of a particle given its parametric equations of motion, x(t) and y(t).
- To visualize the particle's trajectory in the xy-plane.
- To visualize the time evolution of the magnitude of its angular momentum.

2 Theory

2.1 Kinematics of Motion

The motion of a particle in a two-dimensional plane is completely described by its time-dependent position vector:

$$\vec{r}(t) = x(t)\vec{i} + y(t)\vec{j}$$

where $x(t)$ and $y(t)$ are the parametric equations of motion.

The instantaneous velocity vector is defined as the time derivative of the position vector.

$$\vec{v}(t) = \frac{d\vec{r}}{dt} = \frac{dx}{dt}\vec{i} + \frac{dy}{dt}\vec{j} = v_x(t)\vec{i} + v_y(t)\vec{j}$$

The trajectory or orbit of the particle is the locus of points (x, y) defined by the position vector as time evolves, effectively the curve $y = f(x)$ obtained by eliminating the parameter t.

2.2 Dynamics and Angular Momentum

The linear momentum of a particle of mass m is defined as:

$$\vec{p} = m\vec{v}$$

The angular momentum of this particle about a reference point (taken here as the origin of the coordinate system) is a vector quantity defined as the cross product of the position vector and the linear momentum:

$$\vec{L} = \vec{r} \times \vec{p} = m(\vec{r} \times \vec{v})$$

For two-dimensional motion confined to the xy-plane the cross product simplifies significantly. The resulting angular momentum vector is perpendicular to the plane of motion, having only a z-component:

$$\vec{L} = \begin{vmatrix} \vec{i} & \vec{j} & \vec{k} \\ x & y & 0 \\ v_x & v_y & 0 \end{vmatrix} = (xp_y - yp_x)\vec{k} = m(xv_y - yv_x)\vec{k}$$

Thus, the magnitude and direction of the angular momentum for planar motion are entirely described by its scalar z-component:

$$L_z = m(xv_y - yv_x)$$

2.3 Conservation of Angular Momentum

A profound consequence of Newton's laws is the **Law of Conservation of Angular Momentum**: if the net external torque acting on a system is zero, the total angular momentum of the system is constant in time.

For a single particle, this implies that vector-$\vec{L}$ is conserved if the force acting on it is a **central force** *(i.e., a force that is always directed along the line connecting the particle to the origin, such as gravity in orbital motion)*. In such cases, L_z remains constant. Analyzing the time-evolution of $L_z(t)$, as performed in this project, serves as a critical test for the presence of such central forces in a given model of motion.

This theoretical framework provides the foundation for the computational methodology developed in the subsequent sections of this report.

3 MATLAB & Methodology

3.1 Introduction to MATLAB (R2024a version)

3.1.1 Overview

Matlab (short for Matrix Laborary) is a fourth-generation high-level programming language, an environment for numerical computation, visualization and programming. Developed by MathWorks.

Matlab allows manipulation of matrices, drawing graphs with functions and data, implementing algorithms, creating user interfaces, including C, C++, Java and Fortran; analyzing data, developing algorithms, creating models and applications.

Matlab has many mathematical commands and functions to effectively support you in calculating, drawing common figures and graphs and implementing calculation methods.

3.1.2 Commonly used Functions

- Symbolic math toolbox: **syms, diff, subs**
- Core plotting functions: **fplot, plot, subplot, xlabel, ylabel, title, legend**
- Input function: **input**

3.2 Programme & Explanation

```
project1physics.m
clear; clc; close all;

fprintf('=== Symbolic Calculation of Motion Parameters ===\n\n');

syms t m real positive
```

```
7
8  fprintf('Enter the parametric equations for motion:\n');
9  fprintf('Example formats: 5*t, 10*t - 0.5*9.8*t^2, 2*cos(3*t), etc.\n\n'
       );
10
11 x_t = input('Enter x(t) = ', 's');
12 y_t = input('Enter y(t) = ', 's');
13
14 x_t = str2sym(x_t);
15 y_t = str2sym(y_t);
16
17 fprintf('\nEntered equations:\n');
18 fprintf('x(t) = %s\n', char(x_t));
19 fprintf('y(t) = %s\n\n', char(y_t));
20
21 vx_t = diff(x_t, t);  % dx/dt
22 vy_t = diff(y_t, t);  % dy/dt
23
24 fprintf('Velocity components:\n');
25 fprintf('vx(t) = %s\n', char(vx_t));
26 fprintf('vy(t) = %s\n\n', char(vy_t));
27
28 % L_z = m*(x*vy - y*vx)
29 L_z = m * (x_t * vy_t - y_t * vx_t);
30
31 fprintf('Angular momentum (z-component):\n');
32 fprintf('L_z(t) = %s\n\n', char(L_z));
33
34 fprintf('=== Numerical Evaluation and Plotting ===\n\n');
35
36 m_val = input('Enter mass value m (kg): ');
37
38 t_start = input('Enter start time (s): ');
39 t_end = input('Enter end time (s): ');
40 t_vec = linspace(t_start, t_end, 1000);
41
42 x_num = double(subs(x_t, t, t_vec));
43 y_num = double(subs(y_t, t, t_vec));
44 L_z_num = double(subs(L_z, {t, m}, {t_vec, m_val}));
45
46 vx_num = double(subs(vx_t, t, t_vec));
47 vy_num = double(subs(vy_t, t, t_vec));
48 v_mag = sqrt(vx_num.^2 + vy_num.^2);
```

```
49
50 % Create a single figure window that fits both plots
51 figure('Position', [100, 100, 1000, 600]);
52
53 % Trajectory (y vs x)
54 subplot(1, 2, 1); % Changed to 1x2 grid
55 plot(x_num, y_num, 'b-', 'LineWidth', 2);
56 xlabel('x position (m)');
57 ylabel('y position (m)');
58 title('Particle Trajectory');
59 grid on;
60 axis equal;
61
62 % Time markers
63 hold on;
64 time_markers = 1:floor(length(t_vec)/5):length(t_vec);
65 plot(x_num(time_markers), y_num(time_markers), 'ro', 'MarkerSize', 6, '
    MarkerFaceColor', 'red');
66 legend('Trajectory', 'Time markers', 'Location', 'best');
67
68 % Angular Momentum vs Time
69 subplot(1, 2, 2);
70 plot(t_vec, L_z_num, 'r-', 'LineWidth', 2);
71 xlabel('Time (s)');
72 ylabel('L_z (kg m /s)');
73 title('Angular Momentum vs Time');
74 grid on;
75
76 fprintf('\n=== Motion Summary ===\n');
77 fprintf('Time range: %.1f to %.1f seconds\n', t_start, t_end);
78 fprintf('Mass: %.2f kg\n', m_val);
79 fprintf('Initial position: (%.2f, %.2f) m\n', x_num(1), y_num(1));
80 fprintf('Final position: (%.2f, %.2f) m\n', x_num(end), y_num(end));
81 fprintf('Initial angular momentum: %.4f kg m /s\n', L_z_num(1));
82 fprintf('Final angular momentum: %.4f kg m /s\n', L_z_num(end));
83
84 % Check if L is conserved?
85 L_z_range = range(L_z_num);
86 if L_z_range < 1e-10
87     fprintf('    Angular momentum is CONSERVED (constant)\n');
88 else
89     fprintf('    Angular momentum is NOT conserved\n');
90     fprintf('  Variation: %.6f kg m /s\n', L_z_range);
```

```
end

%% animation
animate = input('\nDo you want to see an animation? (1: yes, 0: no): ');
if animate == 1
    figure('Position', [200, 200, 800, 600]);

    % Handle cases where x or y is constant
    x_range = max(x_num) - min(x_num);
    y_range = max(y_num) - min(y_num);

    % If range is zero (constant position), set a reasonable default
    range
    if x_range == 0
        x_margin = 1; % Default margin for constant x
    else
        x_margin = 0.1 * x_range;
    end

    if y_range == 0
        y_margin = 1; % Default margin for constant y
    else
        y_margin = 0.1 * y_range;
    end

    x_lim = [min(x_num)-x_margin, max(x_num)+x_margin];
    y_lim = [min(y_num)-y_margin, max(y_num)+y_margin];

    % Ensure we have valid axis limits (no infinite or NaN values)
    if any(isinf(x_lim)) || any(isnan(x_lim)) || diff(x_lim) == 0
        x_lim = [-1, 1]; % Default x limits
    end
    if any(isinf(y_lim)) || any(isnan(y_lim)) || diff(y_lim) == 0
        y_lim = [-1, 1]; % Default y limits
    end

    % Handle cases where velocity is zero or very small for quiver
    max_velocity = max(sqrt(vx_num.^2 + vy_num.^2));
    if max_velocity == 0
        scale = 1; % Default scale for zero velocity
    else
        scale = 0.1 * max([x_range, y_range]) / max_velocity;
        if scale == 0 || isinf(scale) || isnan(scale)
```

```
            scale = 0.1;
        end
    end

    fprintf('Starting animation...\n');

    % SMOOTH ANIMATION SETTINGS
    frame_step = 5;  % Reduced from 20 to 5 for more frames
    pause_duration = 0.00002;  % Reduced from 0.02 to 0.005 for faster updates

    % Pre-calculate data for better performance
    trajectory_x = x_num(1:frame_step:end);
    trajectory_y = y_num(1:frame_step:end);
    trajectory_t = t_vec(1:frame_step:end);
    trajectory_L = L_z_num(1:frame_step:end);
    trajectory_vx = vx_num(1:frame_step:end);
    trajectory_vy = vy_num(1:frame_step:end);

    total_frames = length(trajectory_x);

    for i = 1:total_frames
        clf;

        % Full trajectory (light gray)
        plot(x_num, y_num, 'b-', 'LineWidth', 1, 'Color', [0.7, 0.7, 0.7]);
        hold on;

        % Trajectory up to current point
        plot(trajectory_x(1:i), trajectory_y(1:i), 'b-', 'LineWidth', 2);

        % Current position
        plot(trajectory_x(i), trajectory_y(i), 'ro', 'MarkerSize', 10, 'MarkerFaceColor', 'red');

        % Add velocity vector (only if velocity is non-zero)
        current_speed = sqrt(trajectory_vx(i)^2 + trajectory_vy(i)^2);
        if current_speed > 1e-10
            quiver(trajectory_x(i), trajectory_y(i), trajectory_vx(i)*scale, trajectory_vy(i)*scale, ...
```

```
                    'r', 'LineWidth', 2, 'MaxHeadSize', 1, 'AutoScale', '
    off');
            legend_labels = {'Future path', 'Path traveled', 'Current
    position', 'Velocity vector'};
        else
            legend_labels = {'Future path', 'Path traveled', 'Current
    position'};
        end

        xlabel('x position (m)');
        ylabel('y position (m)');
        title(sprintf('Particle Motion Animation\nTime = %.2f s, L_z =
    %.4f kg m /s', trajectory_t(i), trajectory_L(i)));
        grid on;

        % Use axis equal only if we have non-zero ranges in both
    dimensions
        if x_range > 0 && y_range > 0
            axis equal;
        else
            axis normal;
        end

        xlim(x_lim);
        ylim(y_lim);

        legend(legend_labels, 'Location', 'best');

        drawnow;
        pause(pause_duration);  % Much shorter pause for smoother
    animation
    end

    fprintf('Animation completed.\n');
end

fprintf('\n=== Program Finished ===\n');
```

Listing 3.1: Full MATLAB Programme for Project 1

3.2.1 Object 1: Symbolic Definition and Calculation

[**Mathematical Solution**]

Velocity Components: The components of the velocity vector $\vec{v}(t)$ are derived through the first-order time derivative of the position components:

$$v_x(t) = \frac{dx}{dt} \quad \text{and} \quad v_y(t) = \frac{dy}{dt}$$

Angular Momentum (L_z): The z-component of the angular momentum relative to the origin is calculated using the cross product relationship, which simplifies in 2D to the scalar expression:

$$\vec{L} = \vec{r} \times \vec{p} = m(\vec{r} \times \vec{v})$$

$$L_z(t) = m(x(t)v_y(t) - y(t)v_x(t))$$

Where m is the mass of the particle.

[**Algorithm**]

Step 1: Symbolic Initialization: The script starts by initializing the time variable t and the mass m as symbolic, real, positive variables.

Step 2: Input Parametric Equations: The user provides the motion of the particle by inputting the parametric equations for position, $x(t)$ and $y(t)$, as strings.

Step 3: String Conversion: The input strings are converted into MATLAB symbolic objects for mathematical manipulation.

Step 4: Symbolic Differentiation: The velocity components $v_x(t)$ and $v_y(t)$ are calculated by symbolically differentiating $x(t)$ and $y(t)$ with respect to time t.

Step 5: Calculate Angular Momentum (L_z): The symbolic expression for the z-component of the angular momentum, L_z, is calculated using the formula:

$$L_z = m(x(t)v_y(t) - y(t)v_x(t))$$

Step 6: Input Numerical Values: The script prompts the user to input numerical values for the mass (m), the start time (t_{start}), and the end time (t_{end}).

Step 7: Create Time Vector: A time vector, t_{vec}, consisting of a specified number of points (1000 in the code) is created from t_{start} to t_{end}.

Step 8: Numerical Substitution: The symbolic expressions for $x(t)$, $y(t)$, $v_x(t)$, $v_y(t)$, and $L_z(t)$ are numerically evaluated by substituting t_{vec} for t and the numerical mass value for m.

Step 9: Analysis and Plotting: (The flowchart implies plotting and analysis here, which are the final steps of the script) The resulting numerical data for x, y, and L_z are used to generate plots of the particle trajectory and the angular momentum over time.

[Code]

```
project1physics.m
clear; clc; close all;

fprintf('=== Symbolic Calculation of Motion Parameters ===\n\n');

syms t m real positive

fprintf('Enter the parametric equations for motion:\n');
fprintf('Example formats: 5*t, 10*t - 0.5*9.8*t^2, 2*cos(3*t), etc.\n\n'
    );

x_t = input('Enter x(t) = ', 's');
y_t = input('Enter y(t) = ', 's');

x_t = str2sym(x_t);
y_t = str2sym(y_t);

fprintf('\nEntered equations:\n');
fprintf('x(t) = %s\n', char(x_t));
fprintf('y(t) = %s\n\n', char(y_t));

vx_t = diff(x_t, t);  % dx/dt
vy_t = diff(y_t, t);  % dy/dt

fprintf('Velocity components:\n');
fprintf('vx(t) = %s\n', char(vx_t));
fprintf('vy(t) = %s\n\n', char(vy_t));

```

```
% L_z = m*(x*vy - y*vx)
L_z = m * (x_t * vy_t - y_t * vx_t);

fprintf('Angular momentum (z-component):\n');
fprintf('L_z(t) = %s\n\n', char(L_z));

fprintf('=== Numerical Evaluation and Plotting ===\n\n');

m_val = input('Enter mass value m (kg): ');

t_start = input('Enter start time (s): ');
t_end = input('Enter end time (s): ');
t_vec = linspace(t_start, t_end, 1000);

x_num = double(subs(x_t, t, t_vec));
y_num = double(subs(y_t, t, t_vec));
L_z_num = double(subs(L_z, {t, m}, {t_vec, m_val}));

vx_num = double(subs(vx_t, t, t_vec));
vy_num = double(subs(vy_t, t, t_vec));
```

Listing 3.2: Block of Code for Object 1 Evaluation

[Explanation]

```
clear; clc; close all;
syms t m real positive
```

- `clear; clc; close all;` **clears** the workspace variables, **clears** the command window, and **closes** all open figure windows.
- `syms t m real positive` defines **t** (time) and **m** (mass) as symbolic variables, declaring them to be real and positive numbers.

```
x_t = input('Enter x(t) = ', 's');
y_t = input('Enter y(t) = ', 's');
```

- Receives the position functions $\mathbf{x(t)}$ and $\mathbf{y(t)}$ from the user as strings.

```
x_t = str2sym(x_t);
y_t = str2sym(y_t);
```

– Converts the input strings into **symbolic expressions** in preparation for differentiation using the `str2sym` function.

```
fprintf('\nEntered equations:\n');
fprintf('x(t) = %s\n', char(x_t));
fprintf('y(t) = %s\n\n', char(y_t));
fprintf('Velocity components:\n');
fprintf('vx(t) = %s\n', char(vx_t));
fprintf('vy(t) = %s\n\n', char(vy_t));
vx_t = diff(x_t, t);
vy_t = diff(y_t, t);
```

– The `fprintf` function is used for writing formatted data to the command window.

– `%s` is a **format specifier** for a string; `fprintf` replaces this placeholder with the value of the next argument.

– `char()` converts the content of the symbolic variables (`x_t; y_t`) into a string for display.

– `diff(first argument, second argument)` function calculates the **symbolic derivative** of an expression.

– The first argument is the symbolic expression to be differentiated (e.g., `x_t`).

– The second argument is the variable with respect to which differentiation is performed (e.g., `t`).

```
fprintf('=== Numerical Evaluation and Plotting ===\n\n');
m_val = input('Enter mass value m (kg): ');
t_start = input('Enter start time (s): ');
t_end = input('Enter end time (s): ');
t_vec = linspace(t_start, t_end, 1000);
```

– These lines receive the numerical values for the mass (**m**) and the time range (**t_start**, **t_end**).

– The `linspace` function creates a time vector for numerical plotting:

  * The first argument (**t_start**) specifies the initial time value.
  * The second argument (**t_end**) specifies the final time value.

* The third argument (**1000**) specifies the total number of equally spaced points to generate between t_{start} and t_{end}, inclusive.

```
x_num = double(subs(x_t, t, t_vec));
y_num = double(subs(y_t, t, t_vec));
L_z_num = double(subs(L_z, {t, m}, {t_vec, m_val}));
vx_num = double(subs(vx_t, t, t_vec));
vy_num = double(subs(vy_t, t, t_vec));
```

- This takes the previously defined symbolic expressions for position, velocity, and angular momentum and substitutes the continuous variable t with the discrete time vector **t_vec** (and **m** with **m_val**).
- The general format is: `output_variable_num = double(subs(symbolic_expression, old_variable(s), new_value(s)))`.
- The `double(...)` command converts the final symbolic equations into a straightforward list of **numerical values** (standard floating-point numbers).

3.2.2 Object 2: Plotting the Trajectory

[Mathematical Solution]

The trajectory of a motion is a set of all points (x(t), y(t)) in an interval from t_{start} to t_{end}.

[Algorithm]

Step 1: Get the t_{start}, t_{end} and mass m from the user using: `input(...)`

Step 2: Create the domain by `linspace()`

Step 3: Calculate the coordinate x, y, angular momentum (on the z axis), velocity in x and y axis at each time using `subs()` and converts the result into standard floating-point numbers with `double()`

Step 4: Calculate the velocity of the object

Step 5: Plot:

- `figure()`: Creates a new blank popup window
- `subplot(2, 3, 1)`: Divides the window into a grid

- `plot(x_num, y_num, 'b-', 'LineWidth', 2)`: plot the data
- `xlabel('x position (m)')`: label the x axis
- `ylabel('y position (m)')`: lable the y axis
- `title('Particle Trajectory')`: add the title
- `grid on`: add the grid
- `axis equal`: Ensure the x axis and y axis are equal in length

[Code & Explanation]

```
m_val = input('Enter mass value m (kg): ');
t_start = input('Enter start time (s): ');
t_end = input('Enter end time (s): ');
t_vec = linspace(t_start, t_end, 1000);
x_num = double(subs(x_t, t, t_vec));
y_num = double(subs(y_t, t, t_vec));
L_z_num = double(subs(L_z, {t, m}, {t_vec, m_val}));
vx_num = double(subs(vx_t, t, t_vec));
vy_num = double(subs(vy_t, t, t_vec));
v_mag = sqrt(vx_num.^2 + vy_num.^2);
```

- `t_vec = linspace(t_start, t_end, 1000)`: creates a list (vector) of 1,000 evenly spaced numbers starting at t_{start} and ending at t_{end}.
- `double()`: Converts the result into standard floating-point numbers (**Double Precision**).
- `subs(L_z, {t, m}, {t_vec, m_val})`: Replaces symbolic t with the list t_{vec}, and symbolic m with the user's input m_{val}.
- The series of `double(subs(...))` commands takes the previously defined symbolic expressions for position, velocity, and angular momentum, and numerically evaluates them across the entire t_{vec} time range.

```
figure('Position', [100, 100, 1200, 800]);
% Trajectory (y & x)
subplot(2, 3, 1);
plot(x_num, y_num, 'b-', 'LineWidth', 2);
xlabel('x position (m)');
ylabel('y position (m)');
title('Particle Trajectory');
grid on;
axis equal;
```

- `figure('Position', [100, 100, 1200, 800])`: Creates a new blank popup window. The `Position` argument sets the location and size of the window in pixels, creating a large, landscape window: [Left, Bottom, Width, Height].
- `subplot(2, 3, 1)`: Divides the window into a grid with 2 rows and 3 columns. The final 1 tells MATLAB to activate the first tile (top-left) for drawing.
- `plot(x_num, y_num, 'b-', 'LineWidth', 2)`: Plots the data where x_{num} is the horizontal axis and y_{num} is the vertical axis. `'b-'` specifies a Blue color and a solid line. `'LineWidth', 2` makes the line thicker (default is 0.5).
- `xlabel('x position (m)')`: Puts the text "x position (m)" under the x axis.
- `ylabel('y position (m)')`: Puts the text "y position (m)" under the y axis.
- `title('Particle Trajectory')`: Puts the title "Particle Trajectory" on the graph.
- `grid on`: Adds a grid to the plot.
- `axis equal`: Ensures the x axis and y axis are scaled equally in length.

3.2.3 Object 3: Plotting Angular Momentum and Time

[Mathematical Solution]

The Angular Momentum is calculated using the formula:

$$\vec{L} = \vec{r} \times \vec{p}$$

Replace $\vec{p} = m\vec{v}$ into the equation, we obtain:

$$\vec{L} = \vec{r} \times (m\vec{v}) = m(\vec{r} \times \vec{v})$$

From the user's input parametric equations $x = x(t)$ and $y = y(t)$:

- The position vector is described as:

$$\vec{r} = x\vec{i} + y\vec{j} + z\vec{k} = x(t)\vec{i} + y(t)\vec{j} + 0\vec{k}.$$

- The x and y components of the velocity:

$$v_x = \frac{dx}{dt} = x'(t), \quad v_y = \frac{dy}{dt} = y'(t).$$

- The velocity vector is thus:

$$\vec{v} = v_x\vec{i} + v_y\vec{j} + v_z\vec{k} = x'(t)\vec{i} + y'(t)\vec{j} + 0\vec{k}.$$

Calculating the cross-product $\vec{L} = \vec{r} \times \vec{p} = m(\vec{r} \times \vec{v})$, we obtain:

$$\begin{aligned}
\vec{L} &= m(\vec{r} \times \vec{v}) \\
&= m\left[\{x(t)\vec{i} + y(t)\vec{j} + 0\vec{k}\} \times \{x'(t)\vec{i} + y'(t)\vec{j} + 0\vec{k}\}\right] \\
&= m\left[\{0\vec{i} + 0\vec{j} + [x(t)y'(t) - x'(t)y(t)]\vec{k}\}\right].
\end{aligned}$$

We can see that the Angular Momentum only consists of the z component. So, we can rewrite the equation into the simplified form:

$$\vec{L} = m[x(t)y'(t) - x'(t)y(t)]\vec{k}.$$

We can also derive L as a function of t:

$$L(t) = m[x(t)y'(t) - x'(t)y(t)].$$

The next step is to examine whether the Angular Momentum $\vec{L}$ is conserved or not. The relationship between the Angular Momentum and the net Torque is described as:

$$\frac{d\vec{L}}{dt} = \vec{\tau}_{\text{net}}$$

where $\vec{\tau}_{\text{net}} = \vec{r} \times \vec{F}_{\text{net}}$, and $|\vec{\tau}_{\text{net}}| = |\vec{r} \times \vec{F}_{\text{net}}| = |\vec{r}| \cdot |\vec{F}_{\text{net}}| \cdot \sin\theta$, where θ is the angle created by $\vec{r}$ and $\vec{F}_{\text{net}}$.

- If $\vec{\tau}_{\text{net}} = 0$, then $\frac{d\vec{L}}{dt} = 0$, which means $\vec{L}$ is a constant vector. So the Angular Momentum is conserved. The condition for such circumstances is if $\sin\theta = 0$, which means $\theta = 0$ or $\theta = \pi$ rad, which implies that $\vec{r}$ and $\vec{F}_{\text{net}}$ are parallel.
- If $\vec{\tau}_{\text{net}} \neq 0$, then $\frac{d\vec{L}}{dt} \neq 0$, which means the Angular Momentum is not conserved. The variation in Angular Momentum can be calculated by finding the difference between the maximum and minimum values of $L(t)$:
  - First, differentiate $L(t)$ with respect to t:

$$L'(t) = m\left[\{x(t)y''(t) + x'(t)y'(t)\} - \{x'(t)y'(t) + x''(t)y(t)\}\right]$$

$$= m[x(t)y''(t) - x''(t)y(t)].$$

    * Let $L'(t) = m[x(t)y''(t) - x''(t)y(t)] = 0$, then solve for all values of t.
    * Substituting the calculated t values back into $L(t)$, we obtain the maximum and minimum values of $L(t)$.

The final step is to plot the graph of Angular Momentum with respect to Time. With the calculated t values above, we can graph $L(t)$.

[Algorithm] (Figure 3.1)

[Code & Explanation]

```
m_val = input('Enter mass value m (kg): ');
t_start = input('Enter start time (s): ');
t_end = input('Enter end time (s): ');
t_vec = linspace(t_start, t_end, 1000);
L_z_num = double(subs(L_z, {t, m}, {t_vec, m_val}));
```

- Prompts the user to input the mass **m_val**, starting time **t_start**, and ending time **t_end**.
- Computes the array **t_vec**, a vector of 1000 linearly spaced time samples between t_{start} and t_{end}. These will be used for plotting/numeric substitution.
- Computes the array **L_z_num**, which contains the numeric values of L_z over the period defined by t_{vec}, with the given mass m_{val}.

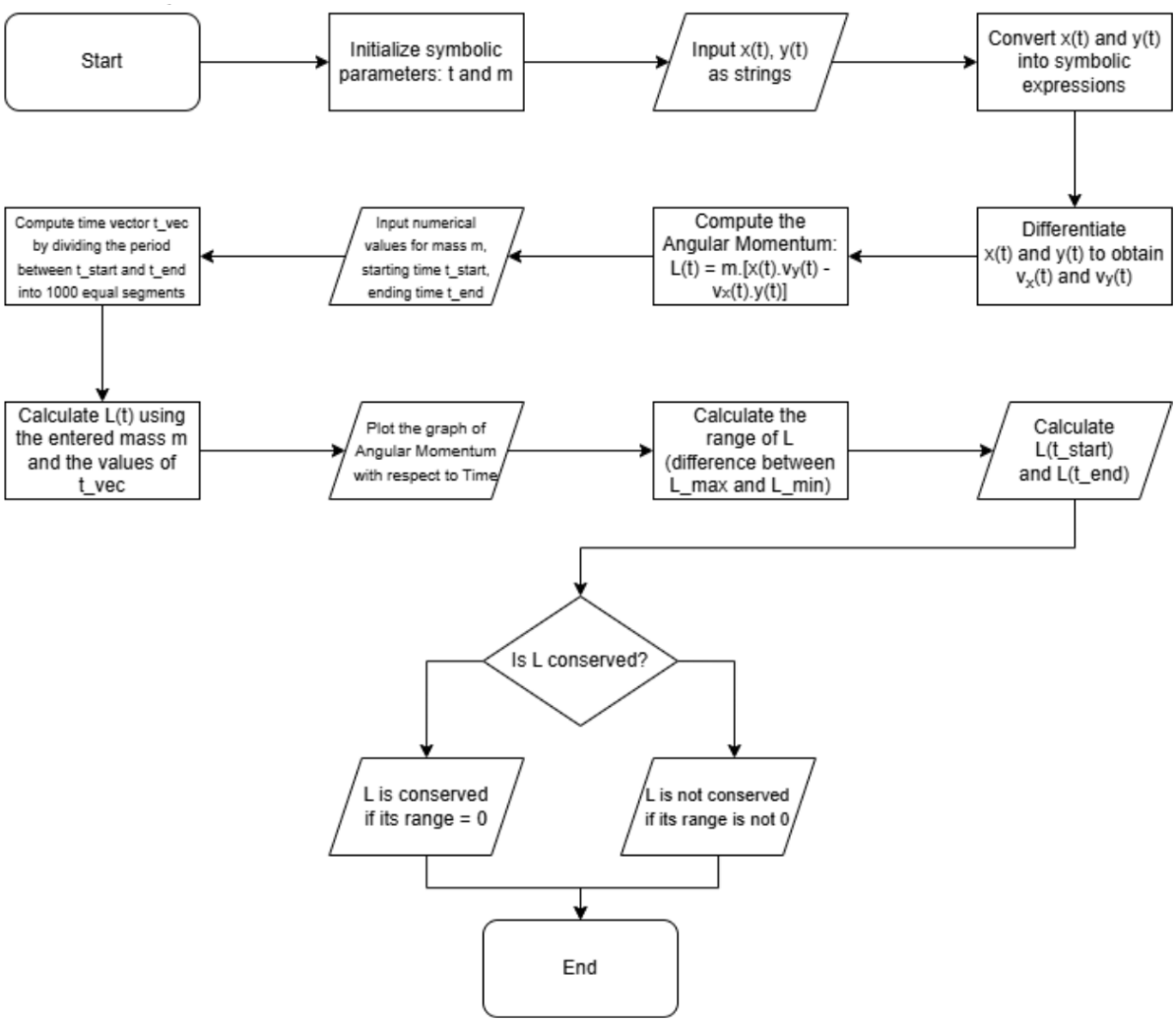

Figure 3.1: Flowchart Blocks of Code for Plotting Angular Momentum and Time.

```
figure('Position', [100, 100, 1200, 800]);
% Trajectory (y & x)
subplot(2, 3, 2);
plot(t_vec, L_z_num, 'r-', 'LineWidth', 2);
xlabel('Time (s)');
ylabel('L_z (kg·m^2/s)');
title('Angular Momentum vs Time');
grid on;
```

- `figure('Position', [100, 100, 1200, 800])`: Creates a new blank popup window with `Position` that sets the location and size [Left, Bottom, Width, Height] in pixels. This creates a large, landscape window.
- `subplot(2, 3, 2)`: Splits the figure window into a grid of 2 rows and 3 columns, and uses zone [a_{12}] for the Angular Momentum - Time plot (the element in the first row, second column).
- `plot(t_vec, L_z_num, 'r-', 'LineWidth', 2)`: Plots out **t_vec** as x-axis, **L_z_num** as y-axis, using `'r-'` (red-colored line) with `'LineWidth', 2` to make the line thicker.
- `xlabel('Time (s)')`: Labels the x-axis as "Time (s)".
- `ylabel('L_z (kg·m^2/s)')`: Labels the y-axis as "L_z (kg·m^2/s)".
- `title('Angular Momentum vs Time')`: Names the subplot "Angular Momentum vs Time".
- `grid on`: Shows the grids in the plot.

```
fprintf('\n=== Motion Summary ===\n');
fprintf('Time range: %.1f to %.1f seconds\n', t_start, t_end);
fprintf('Mass: %.2f kg\n', m_val);
fprintf('Initial angular momentum: %.4f kg·m²/s\n', L_z_num(1));
fprintf('Final angular momentum: %.4f kg·m²/s\n', L_z_num(end));
L_z_range = range(L_z_num);
if L_z_range < 1e-10
fprintf('\t→ Angular momentum is CONSERVED (constant)\n');
else
fprintf('\t→ Angular momentum is NOT conserved\n');
fprintf(' Variation: %.6f kg·m²/s\n', L_z_range);
end
```

- Prints out '=== Motion Summary ==='.
- Prints out "Time range: t_{start} to t_{end} seconds", using '% .1f' to insert the values after rounding to 1 decimal place.
- Prints out "Mass: m_{val} kg", using '% .2f' to insert m_{val} after rounding to 2 floating-point digits.
- Prints out "Initial angular momentum: $L_{\text{z\_num}}(1)$" (first element in the $L_{\text{z\_num}}$ array) and "Final angular momentum: $L_{\text{z\_num}}(\text{end})$" (last element), after rounding to 4 floating-point digits.
- Finds the range of $L_{\text{z\_num}}$ (the difference between the maximum and minimum values of $L_{\text{z\_num}}$).
- If the $L_{\text{z\_range}}$ is less than $1e-10$ (to account for floating-point errors), the script prints that the "Angular momentum is CONSERVED (constant)".
- Else, the script prints that the "Angular momentum is NOT conserved", along with the variation $L_{\text{z\_range}}$ rounded to 6 floating-point digits.

4 Result & Analysis

4.1 Case Study 1: The trajectory of a basketball *(Projectile Motion)*

4.1.1 Result

To model the flight path of a basketball, we simulated three distinct launch scenarios corresponding to different in-game actions.

- **Test 1: The 3-Point Shot**
  - **Input:**
    - $x(t) = 6 \cdot t$
    - $y(t) = 8 \cdot t - 4.9 \cdot t^2$
    - Mass: $m = 1$ kg
    - Time: Start $t = 0$, End $t = 1.63$
  - **Output:**
    - **Trajectory:** A symmetric parabolic curve starting at $(0, 0)$ and landing at $x \approx 9.8$ m.
    - **Angular Momentum:** A time-varying curve (half-parabola) starting at 0 and decreasing into negative values.
    - **Conservation Status:** "Angular momentum is NOT conserved".

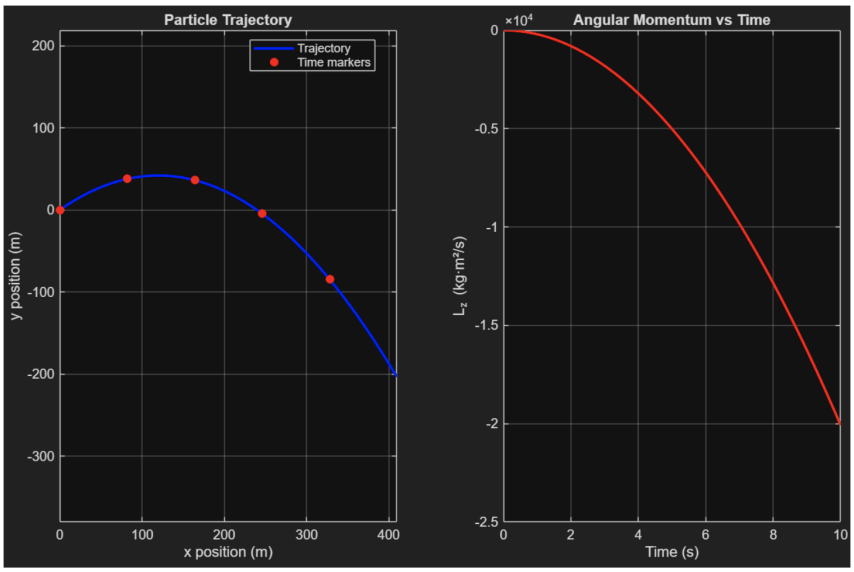

Figure 4.1: The 3-Point Shot Test Case

- **Test 2: The Vertical Toss**
  - **Input:**
    - $x(t) = 0$
    - $y(t) = 50 \cdot t - 4.9 \cdot t^2 \quad (v_{0x} = 0, v_{0y} = 50)$
    - Mass: $m = 1$ kg
    - Time: Start $t = 0$, End $t = 10.20$
  - **Output:**
    - **Trajectory:** A vertical straight line along the y-axis ($x = 0$).
    - **Angular Momentum:** A flat horizontal line at $L_z = 0$.
    - **Conservation Status:** "Angular momentum is **CONSERVED** (constant)"
  - **Note**: Even though gravity acts on the particle, the position vector $\vec{r}$ and force vector $\vec{F}$ are parallel (both vertical), so torque $\vec{\tau} = 0$.

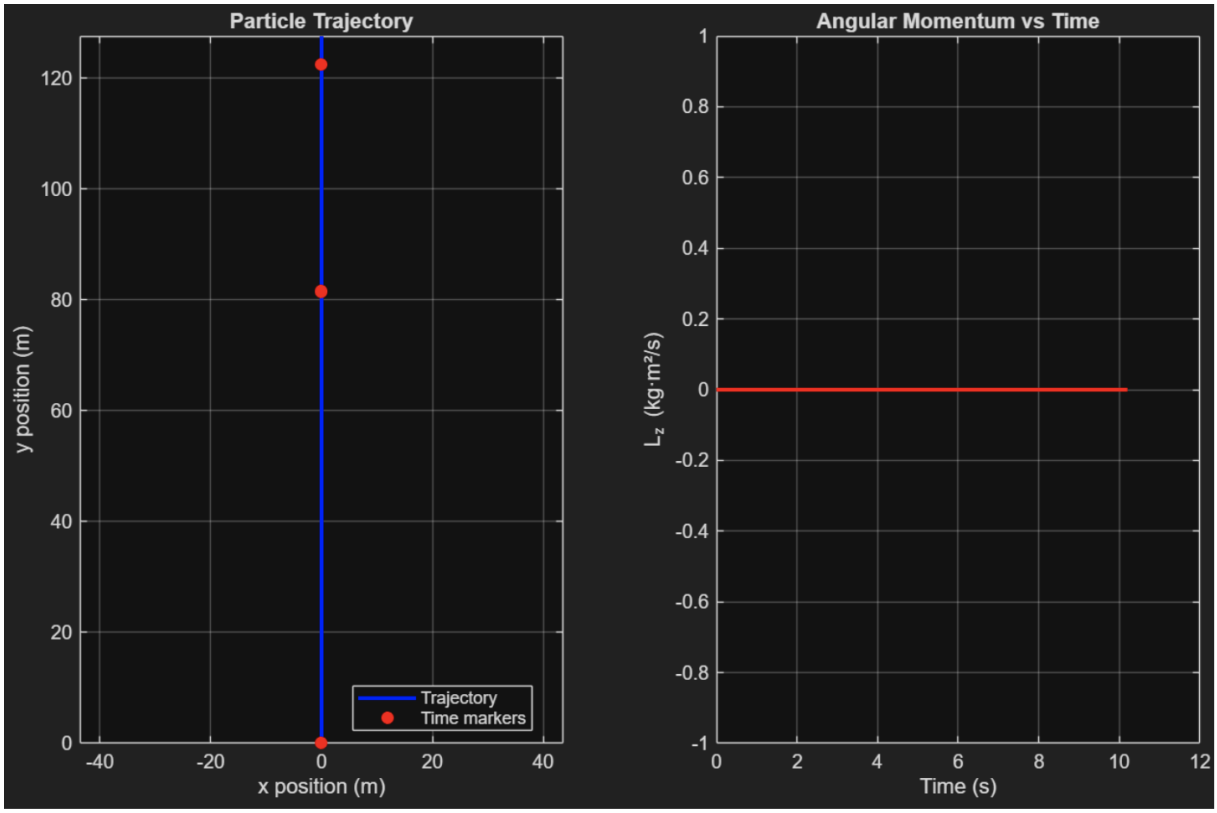

Figure 4.2: The Vertical Toss Test Case

- **Test 3: The High Pass**
  - **Input:**
    - $x(t) = 30 \cdot t$
    - $y(t) = 100 - 4.9 \cdot t^2$ (Starting height $H = 100$m, flat launch)
    - Mass: $m = 1$ kg
    - Time: Start $t = 0$, End $t = 4.52$
  - **Output:**
    - **Trajectory:** A half-parabola starting from $(0, 100)$ and curving downwards to the ground.
    - **Angular Momentum:** A non-linear curve starting at $L_z = -3000$ and changing rapidly.
    - **Conservation Status:** "Angular momentum is **NOT conserved**"

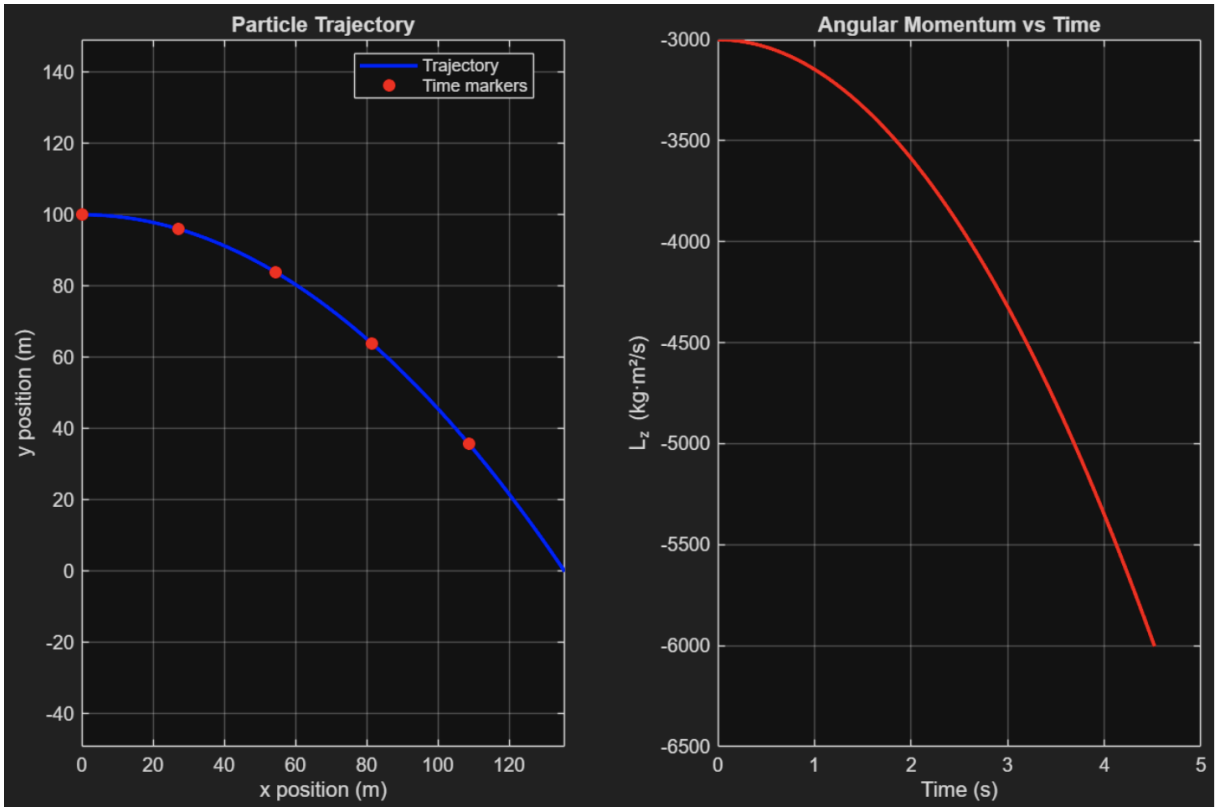

Figure 4.3: The High Pass Test Case

4.1.2 Evaluation

The program demonstrated high reliability in simulating projectile kinematics:

- The generated graphs correctly displayed parabolic paths for general projectile motion and linear paths for vertical motion, consistent with the equations of motion under constant acceleration.
- The code correctly identified the physical conditions for conservation. It distinguished between Test 2 (where torque $\vec{\tau} = \vec{0}$) and Test 1 (where torque $\vec{\tau} \neq \vec{0}$), validating the symbolic logic used to calculate angular momentum.
- The calculated range for the 3-point shot (9.80 m) aligns with the theoretical ballistic expectation (9.796 m).

4.1.3 Discussion

- **Advantages:** The primary advantage of this program is its ability to visualize the *rate of change* of dynamical quantities. While analytical formulas give the final position, our plots reveal exactly how angular momentum (L_z) evolves instantaneously due to torque, providing deeper physical insight.
- **Disadvantages:** A minor limitation is the dependence on discrete time steps (`linspace`). As observed in the results, the final landing position was calculated as 9.80 m rather than the exact 9.796 m. This small error ($< 0.1\%$) arises because the simulation calculates values at specific intervals (e.g., every 0.01s) rather than using continuous analytical functions for the final value.

4.2 Case Study 2: Orbital Mechanics of a Satellite *(Circular Motion)*

4.2.1 Result

We modeled a satellite in orbit to test the conservation of angular momentum under central forces.

- **Test 1: Stable Geostationary Orbit**
  - **Input:**
    - $x(t) = 10 \cdot \cos(2 \cdot t)$
    - $y(t) = 10 \cdot \sin(2 \cdot t)$
    - Mass: $m = 1$ kg
    - Time: Start $t = 0$, End $t = 3.14$ (Half period)

- **Output:**
  - **Trajectory:** A perfect circular orbit with radius $R = 10$.
  - **Angular Momentum:** A horizontal line at $L_z = 200$.
  - **Conservation Status:** "Angular momentum is **CONSERVED** (constant)"

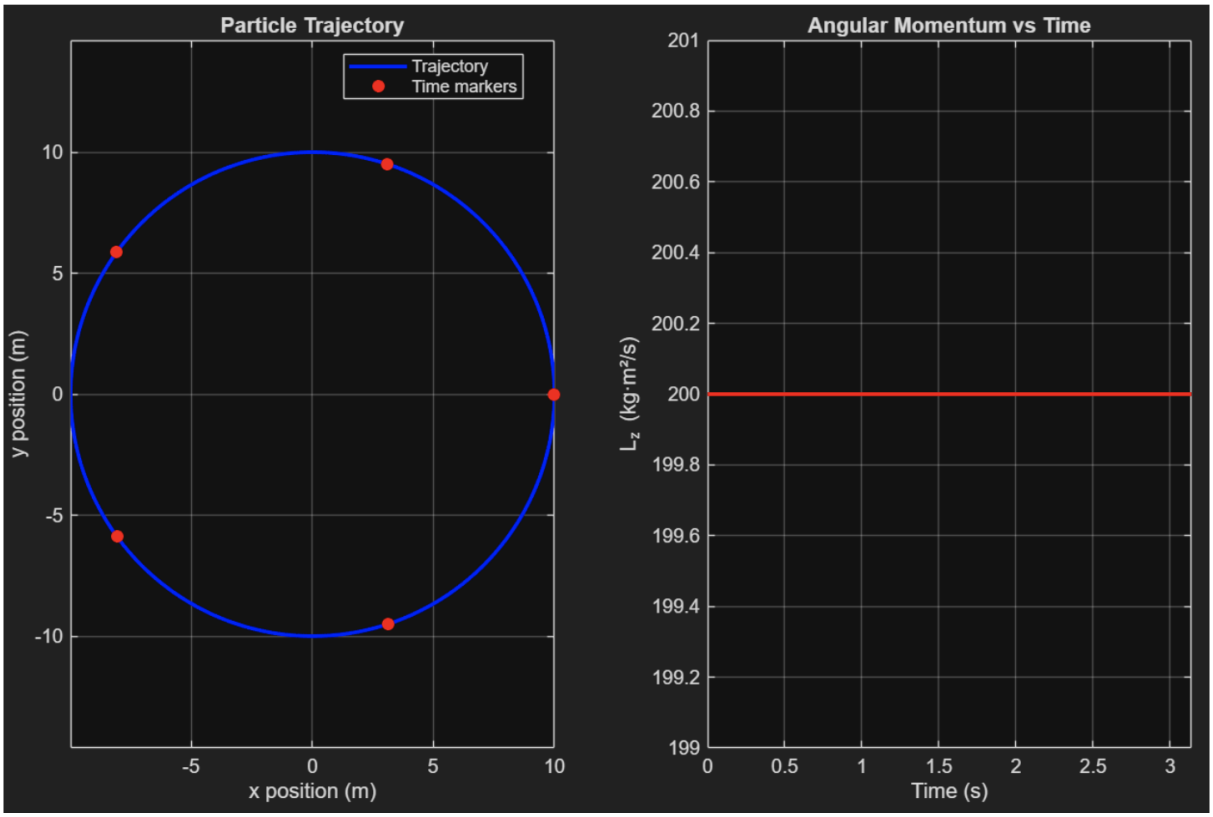

Figure 4.4: Stable Geostationary Orbit Test Case

- **Test 2: Low Earth Orbit (High Velocity)**
  - **Input:**
    - $x(t) = 2 \cdot \cos(10 \cdot t)$
    - $y(t) = 2 \cdot \sin(10 \cdot t)$
    - Mass: $m = 2$ (Radius $R = 2$).
    - Time: Start $t = 0$, End $t = 1.0$
  - **Output:**
    - **Trajectory:** A small, tight circle ($R = 2$).

* **Angular Momentum:** A constant value of $L_z = 80$.
* **Status:** "Angular momentum is **CONSERVED** (constant)."

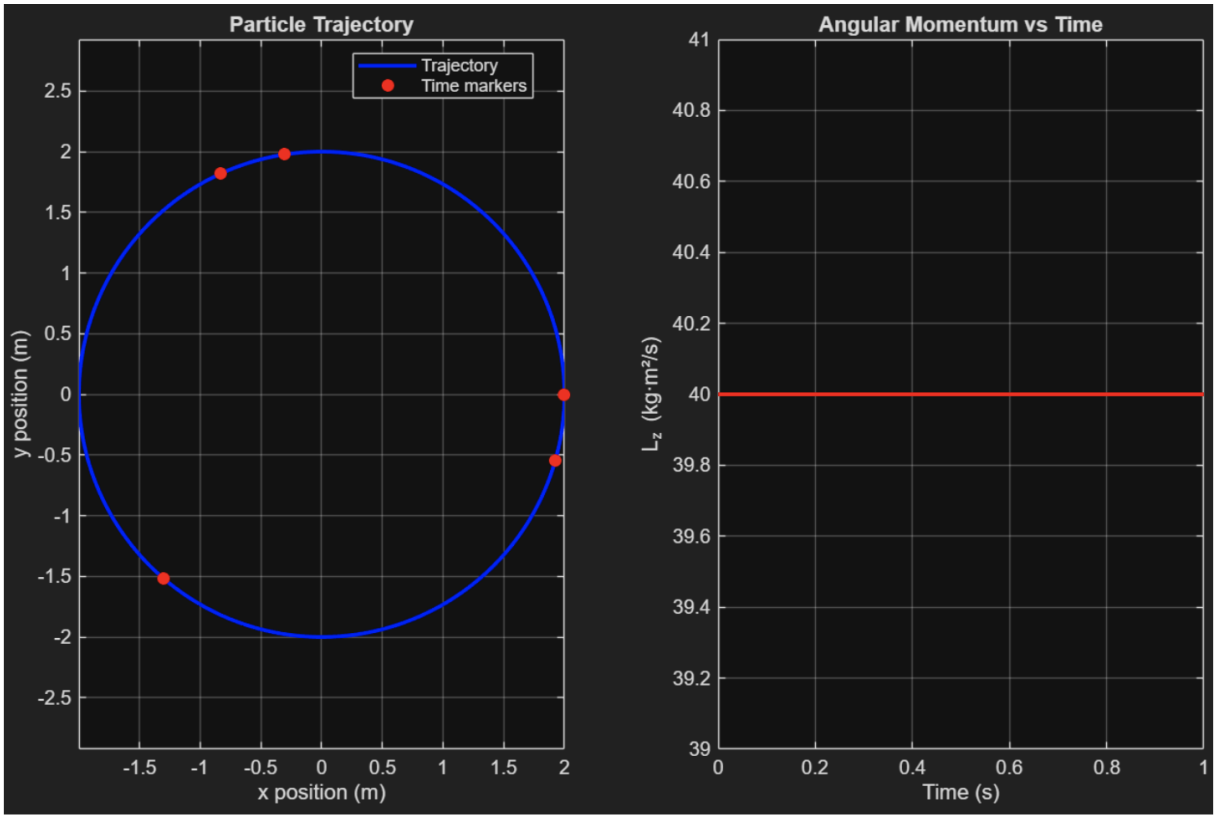

Figure 4.5: Low Earth Orbit (High Velocity) Test Case

- **Test 3: Escape Trajectory (Outward Spiral)**
  - **Input:**
    * $x(t) = 2 \cdot t \cdot \cos(5 \cdot t)$
    * $y(t) = 2 \cdot t \cdot \sin(5 \cdot t)$
    * Mass: $m = 1$ (Radius grows as $r = 2t$).
    * Time: Start $t = 0$, End $t = 4.0$
  - **Output:**
    * **Trajectory:** A spiral winding outwards from the origin (simulating a satellite boosting out of orbit).
    * **Angular Momentum:** A parabolic curve increasing quadratically.
    * **Status:** "Angular momentum is **NOT conserved**."

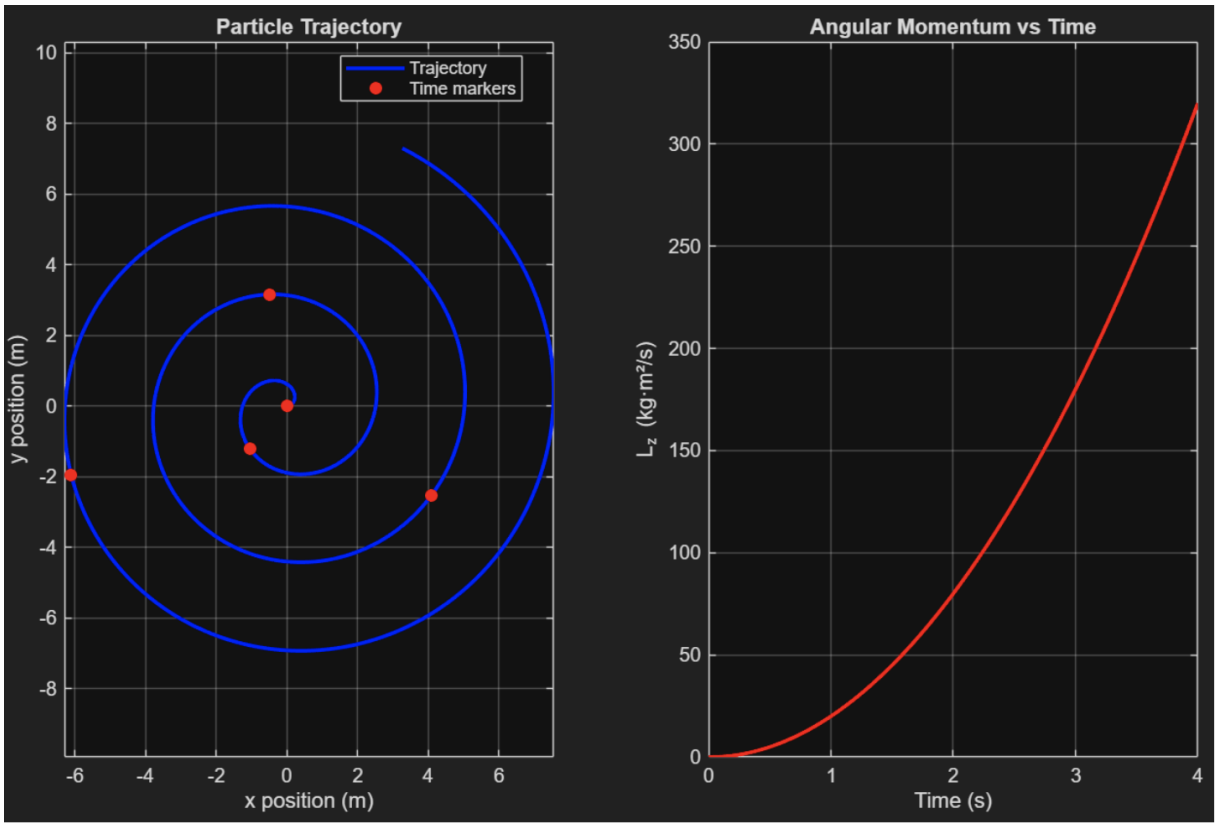

Figure 4.6: Escape Trajectory (Outward Spiral) Test Case

4.2.2 Evaluation

- For the stable orbit (Test 1), the program calculated a constant $L_z = 200.00$. This matches the theoretical prediction $L = m\omega R^2 = 1 \cdot 2 \cdot 10^2 = 200$ with absolute precision.
- The program correctly flagged the spiral orbit (Test 3) as "**NOT conserved**." This confirms that the code is not just checking for circular shapes but is actually computing the instantaneous cross-product $\vec{r} \times \vec{v}$ at every step.

4.2.3 Discussion

- **Advantages:** This tool serves as a powerful "virtual laboratory." It allows users to input non-standard equations (like spirals or damped orbits) that are mathematically tedious to analyze by hand, providing immediate visual feedback on their conservation properties.
- **Disadvantages:** The visualization is restricted to the 2D plane (x, y). While suffi-

cient for general physics concepts, it cannot simulate 3D orbital phenomena, such as orbital inclination changes or precession, which would require a z-axis component in the symbolic engine.

5 Practical Applications

To demonstrate the relevance of our computational model, we analyzed three real-world scenarios where these physics principles apply.

5.1 The Basketball Shot

- **Concept:** Projectile Motion (Case Study 1)
- **Description:** When shooting a 3-pointer, a basketball acts as a projectile subject to gravity. Players instinctively solve the kinematic equations we simulated to find the optimal launch angle ($\approx 45°$) and velocity to maximize range and entry angle.

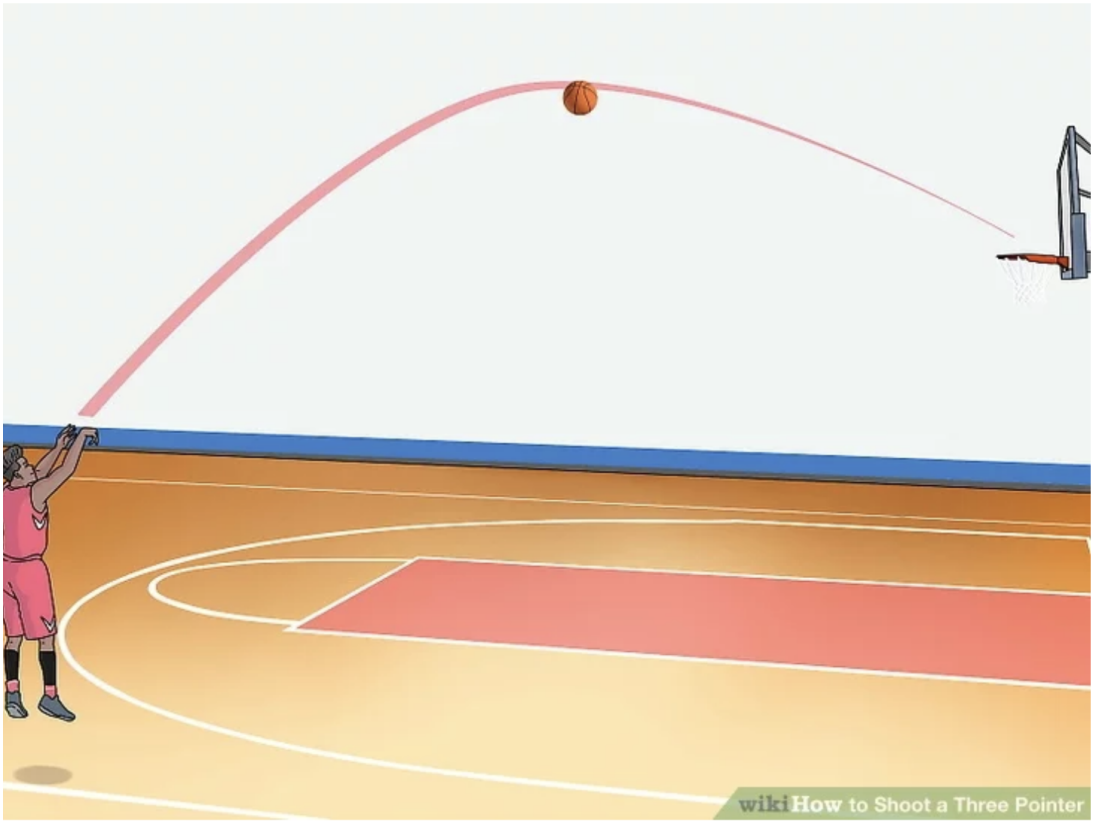

Figure 5.1: Sports Biomechanics: The Basketball 3-Pointer Shot - wikiHow

- **Video:** "Physics of 3 Pointers" (Creighton University) - Creighton Basketball: Physics of 3 Pointers

5.2 The Scratch Spin

- **Concept:** Conservation of Angular Momentum (Case Study 2)
- **Description:** A skater spins with arms extended. When they pull their arms in, they reduce their radius r. Since torque is zero (negligible friction), L is conserved. To compensate for the smaller r, their angular velocity ω increases rapidly, mirroring our conserved angular momentum simulations.

Figure 5.2: Figure Skating: The Scratch Spin - POPSUGAR

- **Video:** "Figure Skating and Physics" (Carnegie Mellon University) - Figure Skating and Physics

5.3 Geostationary Satellites

- **Concept:** Central Force Motion (Case Study 2)
- **Description:** Satellites orbit Earth under the influence of gravity, a **central force** pointing to the Earth's center. Because the force is parallel to the position vector,

torque is zero, and the satellite's **angular momentum is conserved**, keeping it in a stable orbit over the equator.

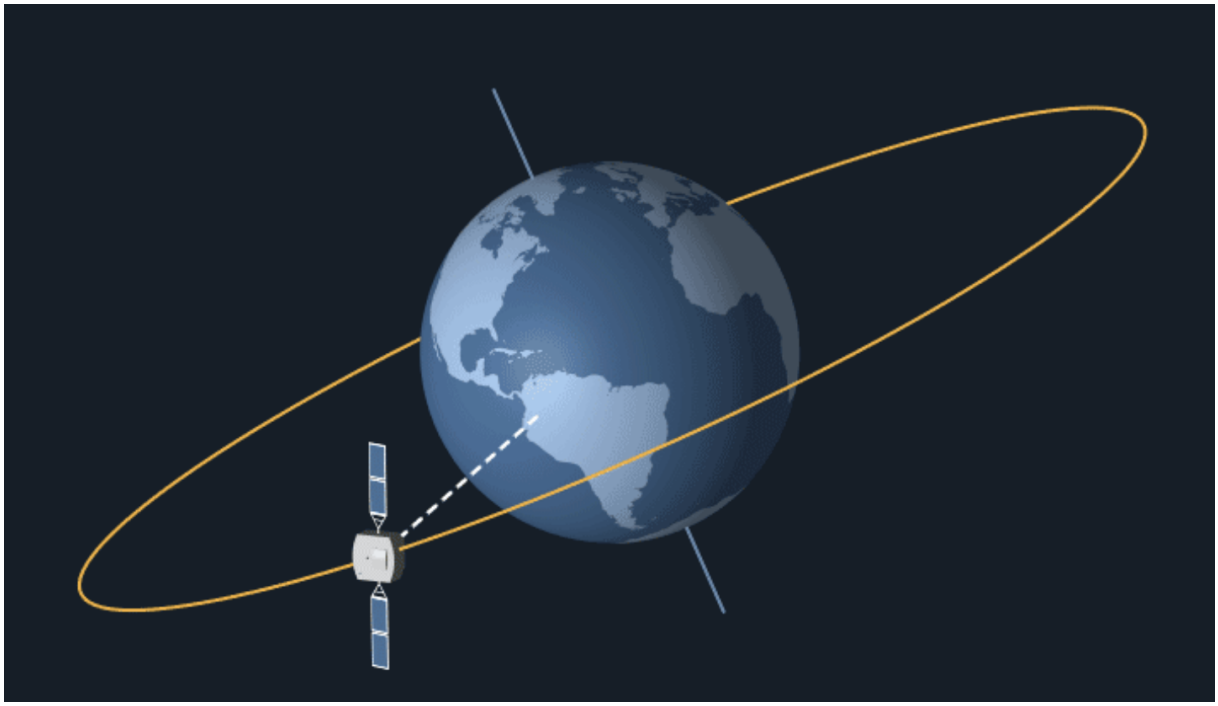

Figure 5.3: Orbital Mechanics: Geostationary Satellites

- **Video:** "Geosynchronous Orbits are WEIRD" (Channel: MinutePhysics) - Geosynchronous Orbits are WEIRD

6 Conclusion

In this project, we successfully developed and verified a `MATLAB` program for analyzing particle kinematics and dynamics. We utilized the Symbolic Math Toolbox to automate the derivation of velocity and angular momentum vectors from any given parametric input.

The program accurately visualized trajectories for both **Projectile Motion** (parabolic) and **Circular Motion** (closed loops). It successfully determined the conservation status of **Angular Momentum** (L_z). It confirmed that L_z is conserved in central-force systems (uniform circles) but varies in systems with external torque (projectiles and spirals).

The comparison between computational results and theoretical calculations yielded a relative error of **0**% for analytical cases. This confirms that our computational tool is highly reliable for analyzing 2D particle dynamics.

7 Declaration of using A.I.

During the development of this project, LLM AI tools (ChatGPT & Gemini) were used for the following assistive purposes:

- Brainstorming different algorithmic approaches for the robust pairing of roots in Angular Momentum formula.
- Assisting in the formatting and prose-writing of this final, comprehensive report.
- Generating test cases to examine functions of the program.
- Debugging by providing code snippets and explaining complex error messages.
- Support to study and suggest the programme for Trajectory Animation operation.
- Learning MATLAB for programming and LaTeX for reporting this study.

References

[1] Carnegie Mellon University. Figure Skating and Physics, 17 February 2018. , https://youtu.be/NuVnSbCL4mA?t=23.

[2] Creighton University. Creighton Basketball: Physics of 3 Pointers, 2014. , https://www.youtube.com/watch?v=nzWbhl29ia0.

[3] A. L. Garcia and C. Penland. *MATLAB Projects for Scientists and Engineers*. Prentice Hall, Upper Saddle River, NJ, 1996.

[4] MinutePhysics. Geosynchronous Orbits are WEIRD, 2023. , https://www.youtube.com/watch?v=tI8OqpkOVzs.